Deevolight

Generic Transaction Screens

Backend API Design — FINAL

FastAPI architecture, contracts, security, permissions, concurrency, lookups and V1 report integration

Final architecture specification — October 2026

1. Purpose and Scope

This document is the implementation specification for the Deevolight Generic Transaction Screen backend. Generic Report Writer V1 is already built and working; this design extends it rather than redesigning it. The immediate goal is a metadata-driven transaction framework whose first proof entity is Lead.

- Backend: Python, FastAPI, SQLAlchemy 2.x, Pydantic v2 and SQL Server 2025.
- Frontend: React/TypeScript consumes frontend-ready screen metadata and entity REST APIs.
- Transaction CRUD is entity-specific. Metadata never generates arbitrary SQL.
- Tenant and user identity come from authenticated backend context, never from client-supplied tenant identifiers.
- FastAPI is the security and validation boundary; React permissions and validation are user-experience aids.

2. Database Prerequisite

One metadata addition is required before implementing lookup rendering: txn.lookup needs display_field_key so the stored return value and user-visible label are explicitly defined.

```
ALTER TABLE txn.lookup
ADD display_field_key varchar(100) NULL;
```

Example: return_field_key = customer_id and display_field_key = customer_name. Application validation must verify that both keys exist among txn.lookup_column.field_key values. NULL is acceptable during development/migration; populated lookup definitions should provide it.

No other database redesign is required for the rules in this document. Tenant-reference validation, permission edge cases, required-field hiding, mass-assignment protection and concurrency behavior are service/API responsibilities.

3. Backend Architecture

```
React
  |
FastAPI Route
  |
Reusable Permission Dependency
  |
Entity / Metadata Service
  |
Repository / SQLAlchemy
  |
SQL Server
```

- Routes are thin: request parsing, dependencies and response mapping.
- Services own business rules, authorization-sensitive decisions, transactions and audit writes.
- Repositories own data access and should make tenant scoping difficult to omit.
- Pydantic/entity services are authoritative for validation; txn.screen_field is a UI rendering/validation hint.
- Entity APIs use hard-coded backend screen keys; the browser never chooses which permission definition protects a route.

4. Tenant Isolation and Cross-Tenant References

Every tenant-owned entity query must include the authenticated tenant_id. A record from another tenant must appear nonexistent to the caller and return 404, not 403.

Foreign keys alone do not prove tenant ownership. Every tenant-owned lookup/reference submitted on POST or PATCH—such as salesperson_id, customer_id or vendor_id—must be validated through the same tenant-scoped logic used by its lookup.

```
Example validation:
salesperson.id = supplied_salesperson_id
AND salesperson.tenant_id = current_tenant_id
AND salesperson.is_deleted = 0
```

Where practical, tenant-owned repositories should inherit or compose a common tenant-scoping mechanism so developers do not have to remember the filter on every query.

5. Validation Authority and Mass Assignment

5.1 Pydantic is authoritative

Pydantic schemas and entity services are the final authority for required fields, types, maximum lengths, defaults and business rules. Metadata communicates those rules to React for rendering and early validation, but cannot weaken backend validation.

Each entity should have automated contract tests comparing important Pydantic constraints with its screen metadata to detect drift.

5.2 Input protection

- Create/update schemas use Pydantic extra="forbid".
- Public input schemas exclude tenant_id, audit columns, deletion flags and other server-owned fields.
- row_version is an opaque concurrency token, not an editable entity property.
- PATCH uses exclude_unset=True so omitted fields mean no change while explicit null means clear the field.
- Attempts to modify readonly fields, or create_only fields after creation, return a validation error keyed by field_key.

6. PermissionService

PermissionService is built before the screen metadata API and is the single implementation of screen/report authorization rules.

6.1 Screen permission resolution

```
No permission row:
  permission_mode = open:
      can_view   = allow_view
      can_add    = allow_add
      can_edit   = allow_edit   AND can_view
      can_delete = allow_delete AND can_view
  permission_mode = restricted:
      deny all

Active permission row:
  can_view   = allow_view   AND can_access AND row.can_view
  can_add    = allow_add    AND can_access AND row.can_add
  can_edit   = allow_edit   AND can_access AND row.can_edit   AND can_view
  can_delete = allow_delete AND can_access AND row.can_delete AND can_view

Inactive permission row (is_active = 0):
  deny all; never fall back to open mode

Deleted permission row:
  historical/deleted; treat as no current permission row
```

Edit and delete require view because both operations load the existing record.

6.2 Reusable FastAPI dependency

```
require_screen_permission("lead", "view")
require_screen_permission("lead", "add")
require_screen_permission("lead", "edit")
require_screen_permission("lead", "delete")
```

The screen key and operation are constants on the backend route. Entity routes do not manually remember to call PermissionService.

7. Screen Metadata API

7.1 GET /api/v1/screens/{screen_key}

This is the primary contract consumed by the generic React transaction renderer.

Step	Responsibility
1	Find active, non-deleted txn.screen by screen_key.
2	Resolve effective permissions for current tenant/user.

Step	Responsibility
3	Return 403 only when all effective operations are false. An Add-only user still needs metadata.
4	Load active sections ordered by display_order.
5	Load fields and merge tenant txn.screen_field_config presentation overrides.
6	Apply required-field visibility safety rule.
7	Attach complete lookup metadata for dropdown fields.
8	Return frontend-ready metadata; do not expose internal database IDs unless required.

7.2 Tenant field merge

```
display_name  = override.display_name  ?? system.display_name
display_order = override.display_order ?? system.display_order
is_displayed  = override.is_displayed  ?? system.is_displayed
```

A required field with no usable default cannot be hidden. The configuration-write service must reject such an override, and metadata generation should defensively ignore an invalid hide override if bad data exists.

Fields whose effective is_displayed is false should normally be omitted from the response.

7.3 Final screen metadata response

```
{
  "screen_key": "lead",
  "display_name": "Lead",
  "layout_type": "standard",
  "api_base_path": "/api/v1/leads",
  "permissions": {
    "can_view": true,
    "can_add": true,
    "can_edit": true,
    "can_delete": false
  },
  "sections": [
    {
      "section_key": "general",
      "display_name": "General",
      "display_order": 10,
      "column_count": 2,
      "is_collapsible": false,
      "is_collapsed_default": false,
      "fields": [
        {
          "field_key": "lead_name",
          "display_name": "Lead Name",
          "data_type": "string",
          "control_type": "text",
          "display_order": 10,
          "column_span": 1,
          "is_required": true,
          "edit_mode": "editable",
          "default_value": null,
          "max_length": 150,
          "min_value": null,
          "max_value": null,
          "format_type": null,
          "help_text": null,
          "lookup": null
        }
      ]
    }
  ]
}
```

Do not return a separate capabilities block. React needs effective permissions; the backend owns the capability/permission calculation.

8. Lookup Metadata and Lookup APIs

8.1 Dropdown metadata

```
{
  "field_key": "salesperson_id",
  "display_name": "Salesperson",
  "data_type": "string",
  "control_type": "dropdown",
  "is_required": true,
  "edit_mode": "editable",
  "lookup": {
    "lookup_key": "salesperson",
    "api_base_path": "/api/v1/lookups/salespersons",
    "return_field_key": "salesperson_id",
    "display_field_key": "salesperson_name",
    "min_search_chars": 2,
    "page_size": 50,
    "columns": [
      {
        "field_key": "salesperson_name",
        "display_name": "Name",
        "display_order": 10,
        "is_displayed": true,
        "is_searchable": true,
        "width": null,
        "format_type": null
      }
    ]
  }
}
```

8.2 Search API

```
GET /api/v1/lookups/customers?search=abc&page=1&page_size=50
```

```
{
  "items": [
    {
      "customer_id": "C1001",
      "customer_name": "ABC Jewelers",
      "city": "New York"
    }
  ],
  "has_more": false
}
```

- Server clamps page_size and enforces lookup.min_search_chars.
- All tenant-owned lookups apply tenant_id and is_deleted filters.
- LIKE search is parameterized and escapes %, _ and [as literal user input where appropriate.
- Every lookup uses deterministic ORDER BY.
- Fetch page_size + 1 rows to calculate has_more.
- Lookup authorization is enforced server-side; lookup availability must not become a route around entity/screen security.

8.3 Existing-ID label resolution

```
GET /api/v1/lookups/customers?ids=C1001&ids=C1002
```

The ids form resolves existing stored values to their display fields without requiring a broad search. It uses the same tenant/security filters as normal lookup search.

9. Entity CRUD Contract - Lead First

The generic screen API describes the UI, but CRUD remains entity-specific. Lead is the first proof implementation.

API	Permission	Responsibility
GET /api/v1/leads/{id}	view	Load tenant-owned Lead and return record envelope.
POST /api/v1/leads	add	Validate, verify references, create, audit and commit.

API	Permission	Responsibility
PATCH /api/v1/leads/{id}	edit	Partial update, edit-mode checks, references, row_version, audit.
DELETE /api/v1/leads/{id}	delete	Concurrency-protected soft delete and audit.

9.1 Record envelope

```
{
  "record": {
    "id": "01J...",
    "lead_name": "ABC Jewelers",
    "company_name": "ABC Jewelers Inc.",
    "salesperson_id": "01J...",
    "email": "buyer@example.com",
    "phone": "408-555-0100"
  },
  "row_version": "AAAAAAAAB9E=",
  "permissions": {
    "can_view": true,
    "can_edit": true,
    "can_delete": false
  }
}
```

row_version is opaque to the client. The permissions block allows record-level restrictions; for example, a converted Lead can become read-only even when the user normally has screen edit permission.

9.2 POST /api/v1/leads

```
require_screen_permission("lead", "add")
  -> Pydantic validation
  -> validate tenant-owned references
  -> LeadService.create()
  -> SQLAlchemy write
  -> audit.audit_log
  -> commit
  -> return created record envelope
```

9.3 PATCH /api/v1/leads/{id}

```
Request:
{
  "record": {
    "lead_name": "ABC Jewelers LLC",
    "email": null
  },
  "row_version": "AAAAAAAAB9E="
}
```

The service rejects readonly/create_only changes, validates references and performs an explicit concurrency update scoped by id, tenant_id and row_version.

```
UPDATE ...
WHERE id = ?
  AND tenant_id = ?
  AND row_version = ?
```

If zero rows are updated, re-select by id + tenant_id. Missing means 404. Present means 409 CONFLICT with reason stale_version. Return the refreshed row_version after a successful update.

9.4 DELETE /api/v1/leads/{id}

DELETE also requires row_version and uses the same 404-versus-stale-409 logic. It is a soft delete: set is_deleted=1, is_active=0, deleted_at and deleted_by. Never hard-delete.

10. Transactions and Audit

The entity service owns the database transaction. Audit and entity changes commit or roll back together.

```
PATCH:
  load/retain old values
```

```
  -> perform concurrency update
  -> write audit row with old/new values
  -> commit

DELETE:
  retain old values
  -> soft delete with row_version
  -> write audit row with old/new state
  -> commit
```

Audit records include tenant, user, entity, entity ID, operation, timestamp and relevant old/new values.

11. Standard Error Contract

```
{
  "code": "CONFLICT",
  "reason": "stale_version",
  "message": "The record was changed by another user.",
  "fields": {}
}
```

HTTP	Code	Use
401	AUTHENTICATION_REQUIRED	Missing/invalid authentication.
403	FORBIDDEN	Authenticated but operation is not allowed.
404	NOT_FOUND	Missing record or record belongs to another tenant.
409	CONFLICT	reason = stale_version, duplicate, or another defined conflict.
422 / 400	VALIDATION_ERROR	Invalid request or fields; preserve V1 fields map.

FastAPI's default 422 response must be mapped to the standard VALIDATION_ERROR envelope so the frontend has one error shape.

12. Report Actions - Extension of V1

12.1 Embed actions in existing report definition

Do not add a second GET solely for actions. Extend the existing V1 report-definition response with actions.

```
{
  "report": { "...": "..." },
  "criteria": [],
  "columns": [],
  "actions": []
}
```

12.2 Resolve action server-side

```
POST /api/v1/reports/{report_key}/actions/{action_key}/resolve
```

The resolver checks source report access, verifies the action belongs to the source report, validates allowed row/criteria source keys, resolves report_action_param, checks target access, and returns trusted navigation instructions.

```
Screen target:
{
  "target_type": "screen",
  "screen_key": "lead",
  "mode": "view",
  "params": { "id": "01J..." }
}
```

```
Report target:
{
  "target_type": "report",
  "report_key": "sales_detail",
  "criteria": {
    "...": "resolved using the target report's actual V1 criterion contract"
  }
```

```
}
```

Report-to-report examples and implementation must use the actual V1 criterion structure, including the existing single daterange criterion where applicable, rather than inventing from_date/to_date criteria.

13. Apply Permissions to Existing V1 Endpoints

Report permission handling becomes an explicit V2 backend task. Existing V1 report-definition, execution and relevant saved-view/action endpoints must call the centralized report permission resolver.

Regression tests must verify that open reports continue to behave as before while restricted reports and explicit permission rows behave according to the new model.

14. Startup / Deployment Metadata Validation

A validation step should detect broken metadata before users encounter it. This can run at application startup in development and as a deployment validation in production.

- Every active txn.screen.api_base_path corresponds to an expected registered entity API.
- Every active txn.lookup.api_base_path corresponds to an expected registered lookup API.
- lookup.return_field_key exists in txn.lookup_column.
- lookup.display_field_key exists in txn.lookup_column.
- screen_field.section_id belongs to the same screen as screen_field.screen_id.
- Dropdown fields reference valid active lookups.
- Report action source/target keys are valid for their configured reports/screens where validation can be performed.

15. Final Backend Build Order

Order	Deliverable	Completion objective
1	PermissionService + FastAPI dependencies	One authoritative implementation for screen/report permissions.
2	Screen Metadata API	GET /screens/{screen_key} returns complete frontend-ready metadata.
3	First required Lead lookup	Search, paging, tenant filtering and ID-to-label resolution.
4	Lead GET	Record envelope, row_version and record-level permissions.
5	Lead POST	Validation, reference security, audit and transaction.
6	Lead PATCH	Partial update, edit-mode rules, row_version and audit.
7	Lead DELETE	Permission, row_version, soft delete and audit.
8	React generic transaction renderer	Consumes finalized metadata/entity contracts.
9	V1 report definition actions	Embed actions in existing report definition response.
10	Report -> Screen resolve	Secure Lead List -> Lead navigation.
11	Report -> Report drilldown	Reuse existing V1 execution with resolved target criteria.

16. Acceptance Criteria for Backend Foundation

- GET /api/v1/screens/lead returns complete metadata for an Add-only, View/Edit and restricted user correctly.
- Tenant overrides merge correctly and cannot hide an unusable required field.
- All entity reads/writes are tenant-scoped; another tenant's record returns 404.
- Tenant-owned references cannot be assigned across tenants.

- Input schemas reject extra/server-owned fields and invalid edit-mode changes.
- Lookup APIs enforce search/paging/security rules and resolve existing IDs to labels.
- Lead GET/POST/PATCH/DELETE use the standardized envelope and error contract.
- PATCH and DELETE enforce explicit SQL row_version concurrency and distinguish 404 from stale 409.
- Audit writes and entity writes share one service-owned transaction.
- Existing V1 reports continue working while report permissions/actions are added incrementally.
- Customer can later reuse the generic metadata framework with entity-specific CRUD rather than backend redesign.